CS102 – Algorithms and Programming II
Programming Assignment 5
Fall 2025-2026

ATTENTION:

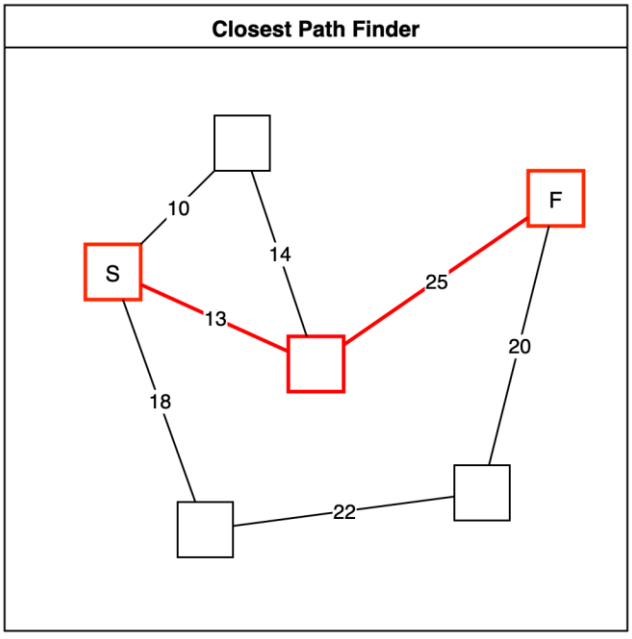
- Compress all of the Java program source files (.java) files into a single zip file.
- The name of the zip file should follow the below convention, where you replace the variables “Sec1”, “Surname” and “Name” with your actual section, surname and name:
  CS102_Sec1_Asst5_Surname_Name.zip
- Upload the above zip file to Moodle by the deadline (if not, significant points will be taken off). You will get a chance to update and improve your solution by consulting to the TAs and tutors during the lab. You have to make the preliminary submission before the lab day. After the TA checks your work in the lab, you will make your final submission. Even if your code does not change in between these two versions, you should make two submissions for each assignment.

GRADING WARNING:

- Please read the grading criteria provided on Moodle. The work must be done individually. Code sharing is strictly forbidden. We are using sophisticated tools to check the code similarities. The Honor Code specifies what you can and cannot do. Breaking the rules will result in disciplinary action.

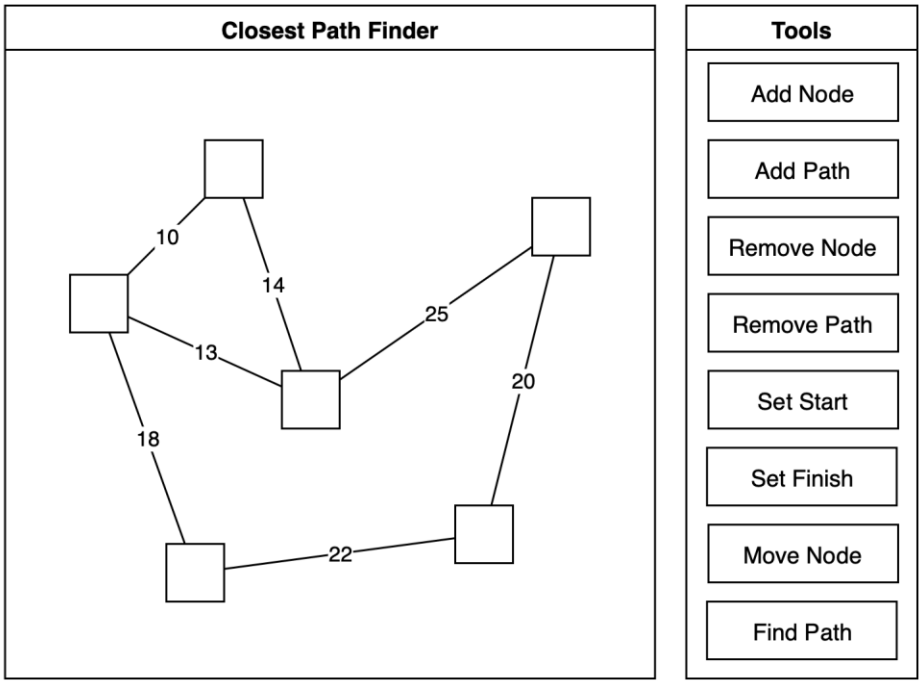
PATH FINDING IN A BIDIRECTIONAL GRAPH

For this lab, you will implement a GUI application that finds the closest path in a bidirectional graph representing paths between locations. The user should be able to add nodes and paths between nodes. Squares will represent nodes, and lines between nodes will represent paths. Once you calculate the shortest path between the start (S) and finish (F) nodes, you will highlight the path and any nodes along the path.



Each node should keep its 2D position in space. You should have a 500 by 500 pixel panel to draw the graph. The length of a path is calculated based on the Euclidean distance between its two ends. While finding the closest path from the start node to the finish node, we can only travel through the connected nodes. We want to find the path whose total length is the smallest.

The application should include two frames. In one frame, you should display the current graph. In the second screen, you will include buttons supporting various functions. An example of these windows is given below.



The “Add Node” button switches the current mouse mode to add new nodes on the screen. The mouse's left click should add a new node, given that another node does not occupy the location. Each node is 20 by 20 pixels. The left click should do nothing if the mouse's current position is already within an existing node. For this purpose, you should iterate over all the nodes and check if the position is included.

The “Add Path” button switches the current mouse mode to add paths between existing nodes. In this mode, the user clicks on two nodes one-by-one to form a path between them. You should highlight the selected nodes using color to make the application more interactive. After selecting this mode, the first node we click on is chosen as A, and the second node is selected as B. When both A and B are selected, create a path between these two nodes if not created before. At any time, if we click on an empty area, both A and B are deselected. Similarly, when the new path is formed, both A and B are deselected so that we can choose new nodes to form a different path. Any path created should inform the connected nodes so the path can be used in both directions while searching for the closest path. The distance of the path should be calculated based on the Euclidean distance between A and B. Round this distance to the closest integer so that its visualization is simpler. Every path should show its distance.

The “Remove Node” button should switch the current mouse mode so that the nodes clicked are deleted. You should also delete the paths that are connected to the deleted node. The “Remove Path” button switches the mouse mode to delete the paths that connect the clicked pair of nodes. This function would work similarly to adding a new path. We first click on some node A, then the second node we click on is marked as B. If a path exists between nodes A and B, it is removed. In this case, do not forget to remove the reference to the path

from both A and B. If there is no such path between A and B, do nothing. At any time, if we click on an empty area, A and B are deselected. Similarly, once we remove a path, both A and B are deselected so that we can choose new pairs.

The “Set Start” button changes the mouse mode so that any node we click is set as the starting node. The starting node should display the character ‘S’. There can be only one starting node. Similarly, the “Set Finish” node switches the mouse mode to set the node we click on as the finishing node. This node should display the letter ‘F’ on it. Again, there can only be one finishing node, so the last node we click stays as the finishing node.

The “Move Node” button switches the mouse mode so that we can move existing nodes by dragging them with the mouse. The mouse button should be pressed on top of a node to move it, and then, until the mouse button is released, the currently selected node follows the mouse position. Releasing the mouse button should fix the position of the node. Note that, while the node updates its position, any path it is connected to should also be updated. This includes redrawing the path line and recalculating its distance based on the current positions of the nodes the line connects.

The “Find Path” button should be disabled unless there is one start and one finish node. This button becomes active after the user sets one start and one finish node. If a start or finish node is removed, the “Find Path” button becomes disabled again. Clicking on this button should find the shortest distance path between the start and finish nodes. If there is no such path (for example, if start and finish nodes are disconnected), display an error message. If such a path exists, highlight the nodes and lines that contribute to this shortest path using a different color. Note that highlighting one of them is sufficient if there are multiple shortest paths with the same total distance. Altering the current graph by inserting, removing, or moving nodes or paths resets the highlighted portion.

In order to find the shortest path, you need to traverse the nodes of the graph, following the connections. This requires you to store the paths connected to a specific node. These paths should also store a reference to the nodes they connect. You may include additional variables in nodes and paths to track the shortest distance traversed so far. Also, you may keep a Boolean variable to track whether a node or path is visited in this traversal. You must also track which nodes and paths are used to achieve the shortest distance. For example, if a specific node can be reached by traversing a shorter distance, you need to update the path from which this node should be visited.

Suppose you keep a reference to the previous node to visit for each node, which achieves the shortest distance. Then, following these nodes in the reverse direction, starting with the finish node, should reveal the shortest path, if such a path exists. During traversal, if we cannot reach the finish node and all the nodes are already visited, no such paths exist between the start and end nodes.

Preliminary Submission: You will submit an early version of your solution before the final submission. This version should at least include the following:

- You should complete the GUI by adding and removing nodes and paths so that they are functional.
- We also suggest that you start the algorithm to find the shortest path.

Not completing the preliminary submission on time results in 50% reduction of this assignment’s final grade.